LAPORAN PRAKTIKUM

IOT 2

Disusun oleh:

Nama : Salim Suheri Sitompul

NPM : 25782053

Kelas : TRI 3B

Dosen:

Dr.Ir.Septafiansyah Dwi Putra,S.T.,M.T.

Agus Ambarwari, S.Pd.,M.Kom.

Andi Saptono,S.Kom

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026

7.5.1 Praktikum 1: Pembangunan Ekosistem IoT Server dengan Docker Compose

- Struktur direktori

```
D:\Docker\iot_server>tree
Folder PATH listing
Volume serial number is 6841-F34B
D:.
├───mosquitto
│   ├───config
│   ├───data
│   └───log
└───nodered
    ├───lib
    │   └───flows
    └───node_modules

D:\Docker\iot_server>
```

- Masuk ke direktori iot-server

```
D:\Docker\iot_server>
```

- Menunggu kedua kontainer sampai selesai
- Proses selesai

```
D:\Docker\iot_server>docker compose up -d
[+] up 2/2
 ✔Container mqtt-broker Running                                                    0.0s
 ✔Container node-red    Running                                                    0.0s

D:\Docker\iot_server>
```

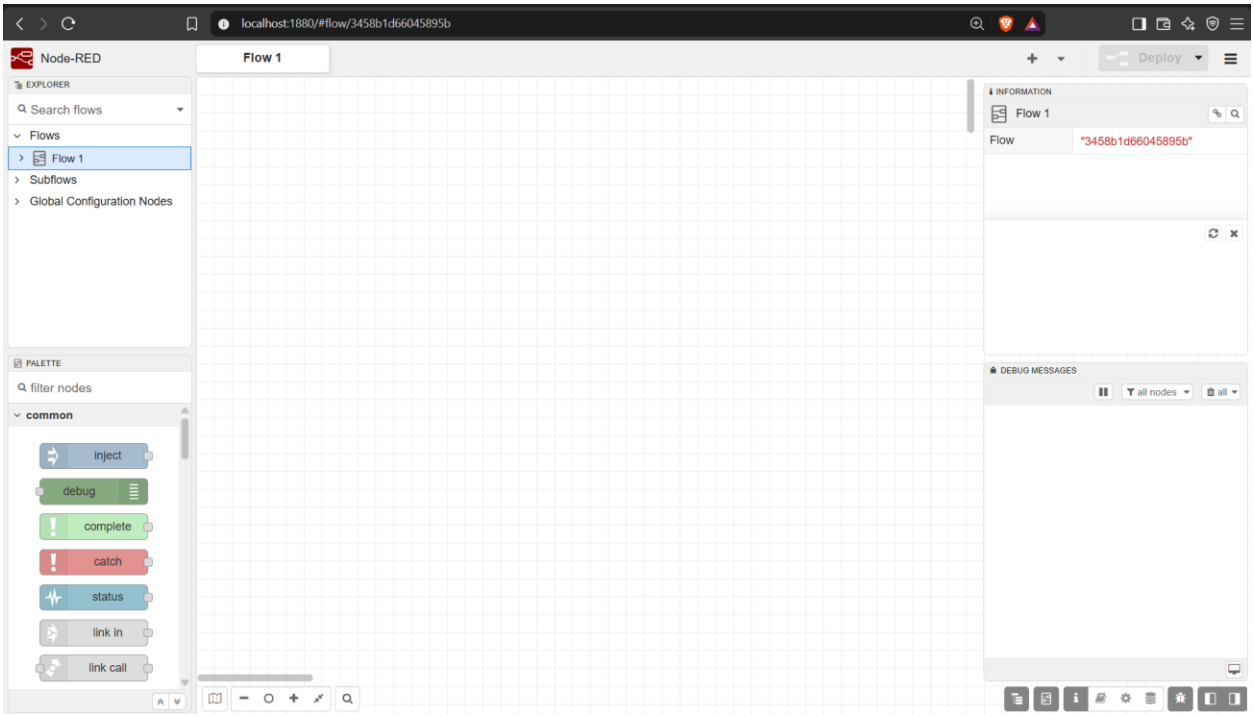
- Verifikasi kedua kontainer telah aktif dan berjalan normal

```
D:\Docker\iot_server>docker ps
CONTAINER ID   IMAGE                     COMMAND                  CREATED      STATUS                    PORTS
                                                                                NAMES
cb3deb66ea26   nodered/node-red:latest    "./entrypoint.sh"        5 days ago   Up 36 seconds (healthy)   0.0.0.0:1880->
1880/tcp, [::]:1880->1880/tcp                                                   node-red
8428a7f17741   eclipse-mosquitto:latest   "/docker-entrypoint.…"   5 days ago   Up 38 seconds             0.0.0.0:1883->
1883/tcp, [::]:1883->1883/tcp, 0.0.0.0:9001->9001/tcp, [::]:9001->9001/tcp   mqtt-broker

D:\Docker\iot_server>
```

STATUS untuk kontainer mqtt-broker dan node-red menunjukkan Up (healthy/running).

- server IoT lokal telah siap beroperasi secara sempurna!



Kesimpulan:

Praktikum ini berhasil membangun infrastruktur server IoT lokal yang terintegrasi secara otomatis menggunakan Docker Compose, dengan menggabungkan Eclipse Mosquitto sebagai broker MQTT dan Node-RED sebagai visual rule engine di dalam satu lingkungan jaringan virtual yang sama. Broker Mosquitto dikonfigurasi menggunakan port 1883 untuk komunikasi data MQTT murni dari mikrokontroler seperti NodeMCU ESP8266 serta port 9001 untuk protokol WebSockets. Melalui berkas deklaratif docker-compose.yml, pengelolaan siklus hidup layanan seperti manajemen volume, kebijakan restart, dan port forwarding dapat dilakukan dengan mudah. Dengan berjalannya kontainer Mosquitto dan akses antarmuka Node-RED melalui http://localhost:1880, fondasi server IoT lokal kini telah siap sepenuhnya untuk memproses data telemetri serta menjalankan otomasi kendali perangkat keras secara real-time.

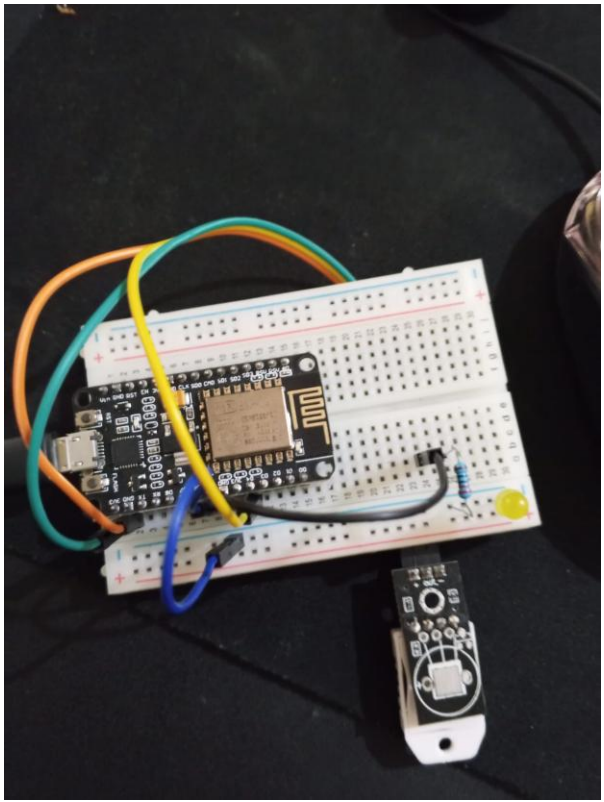
7.5.2 Praktikum 2: Migrasi Simpul NodeMCU ESP8266 ke Broker MQTT Lokal

Sensor DHT11/DHT22:

- Pin VCC → Pin 3V3 NodeMCU
- Pin GND → Pin GND NodeMCU
- Pin Data → Pin D4 (GPIO 2)

LED Indikator Aktuator:

- Anoda (kaki panjang) → Pin D1 (GPIO 5)
- Katoda (kaki pendek) → Resistor 220 Ω → Pin GND



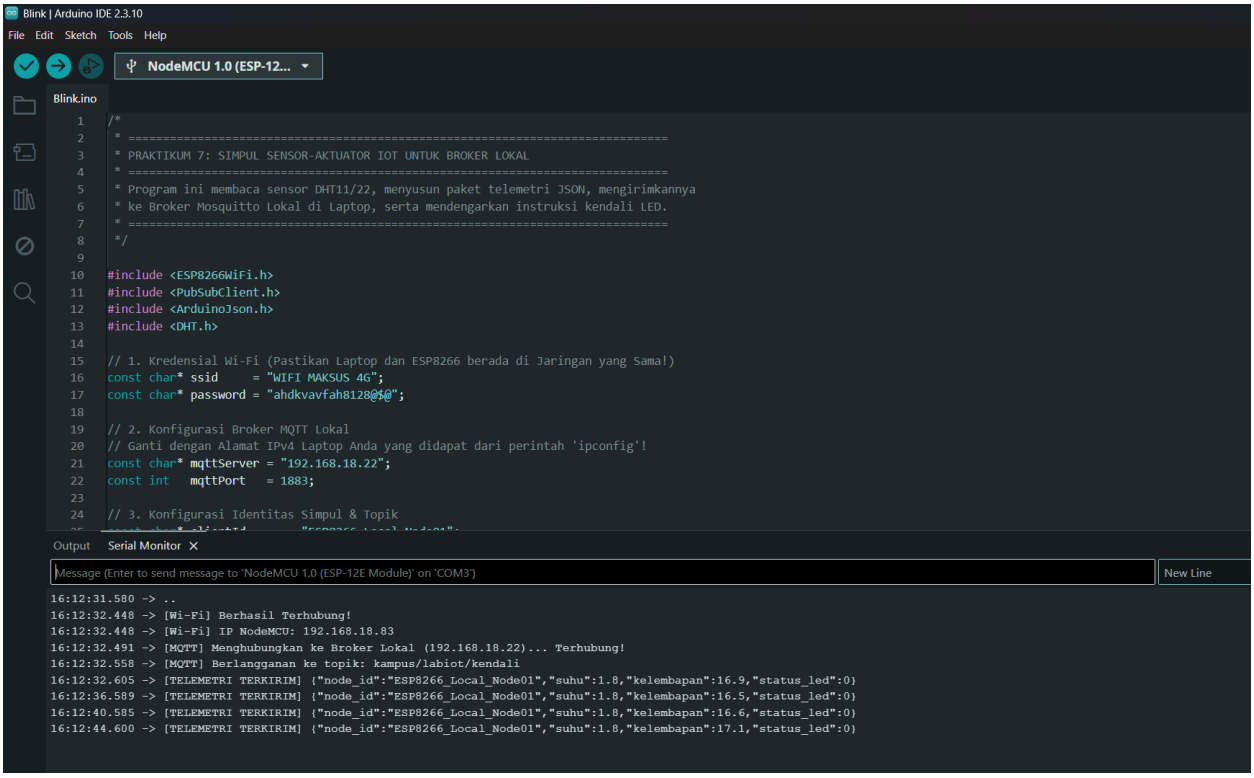
- Ip dari ipconfig

```
Wireless LAN adapter Wi-Fi:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::eb9d:70f3:1e4b:94fb%9
   IPv4 Address. . . . . . . . . . . : 192.168.18.22
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : 192.168.18.1

C:\Users\Salim>
```

- Hasil



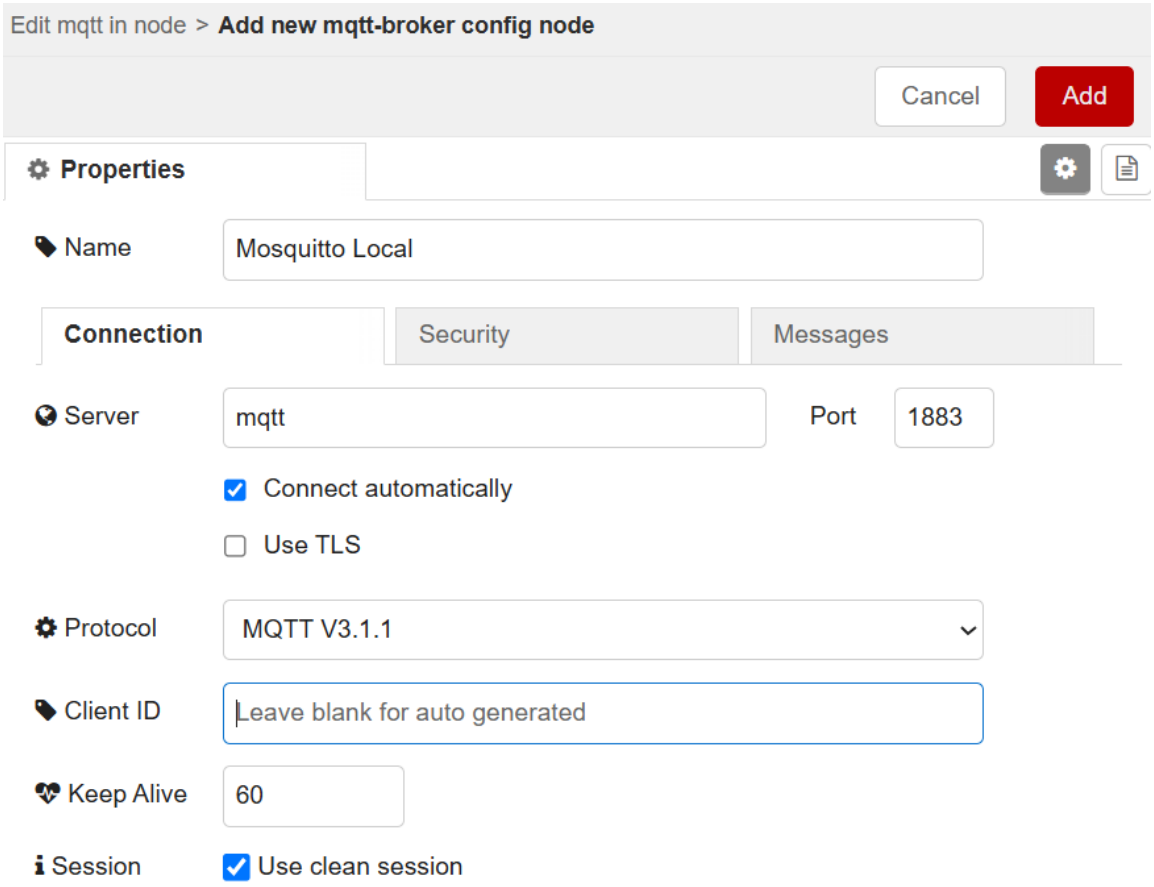
Kesimpulan:

Praktikum ini berfokus pada migrasi simpul NodeMCU ESP8266 agar beralih dari penggunaan broker publik menuju broker MQTT lokal yang ada di laptop. NodeMCU ESP8266 dikonfigurasi untuk terhubung ke jaringan Wi-Fi yang sama dengan laptop dan diarahkan ke alamat IPv4 lokal laptop menggunakan port MQTT standar 1883. Perangkat keras menggunakan sensor DHT11/DHT22 pada pin D4 untuk membaca suhu dan kelembapan, serta indikator LED pada pin D1 yang dikendalikan secara jarak jauh melalui paket data JSON. Pengaturan keamanan *Windows*

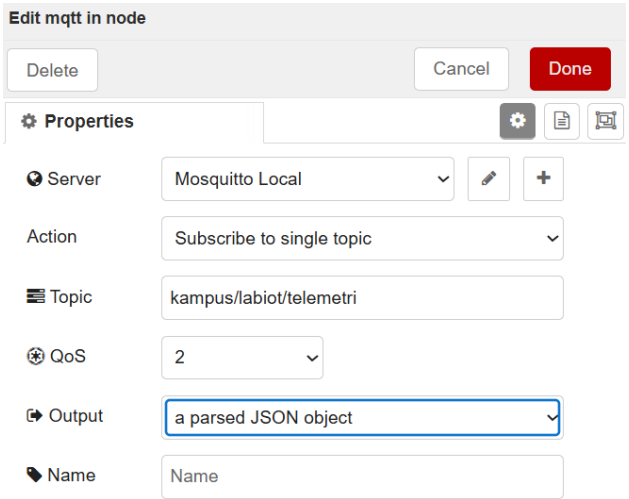
Defender Firewall juga diperlukan untuk mengizinkan koneksi masuk pada port 1883 agar komunikasi antara mikrokontroler dan broker lokal di laptop tidak terblokir. Melalui program yang diterapkan, mikrokontroler dapat mengirim paket data telemetri secara periodik setiap 4 detik sekaligus mendengarkan instruksi kendali dari topik MQTT yang disubskripsikan.

7.5.3 Praktikum 3: Integrasi Alur Data Telemetri Dua Arah pada Node-RED

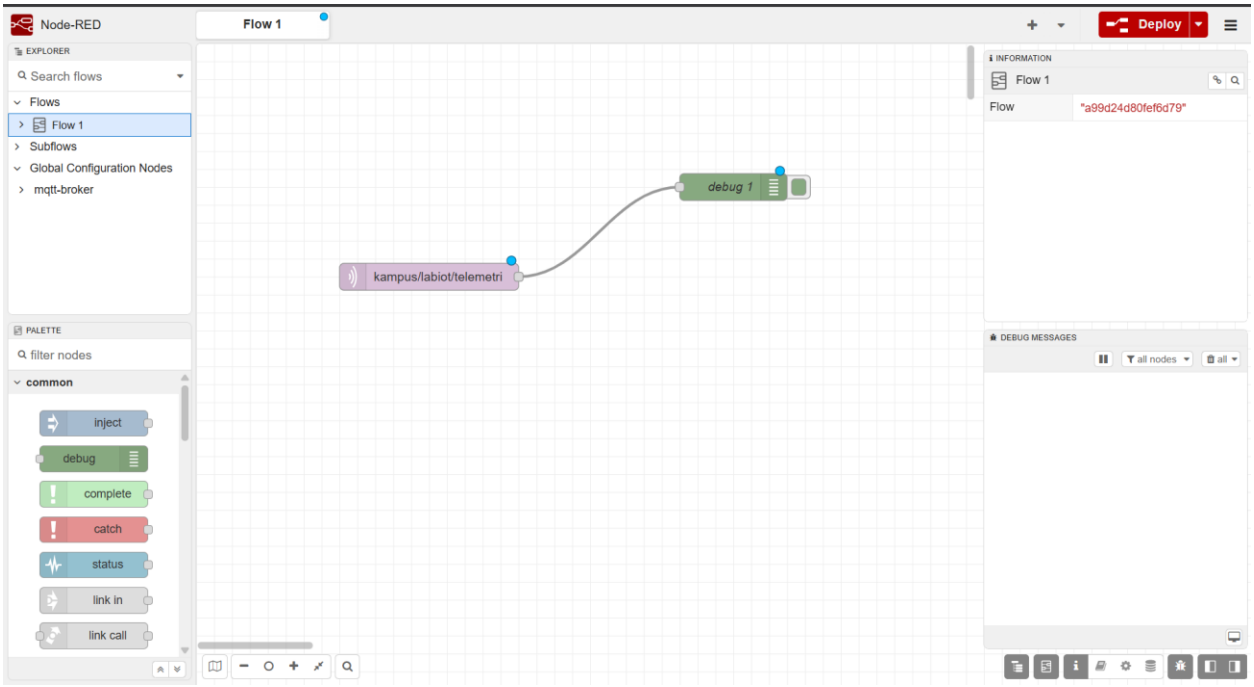
Membuat Konfigurasi Server MQTT Broker



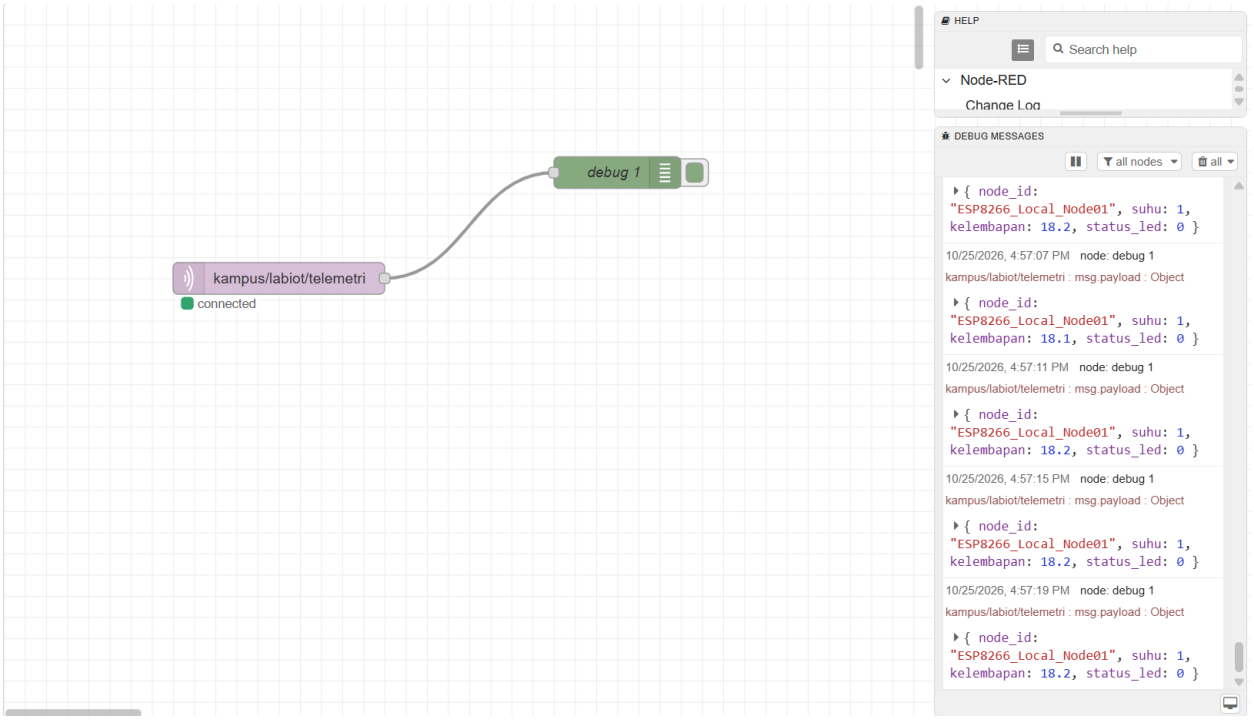
Konfigurasi Parameter Topik



Membuat Alur Kerja (Deploy)



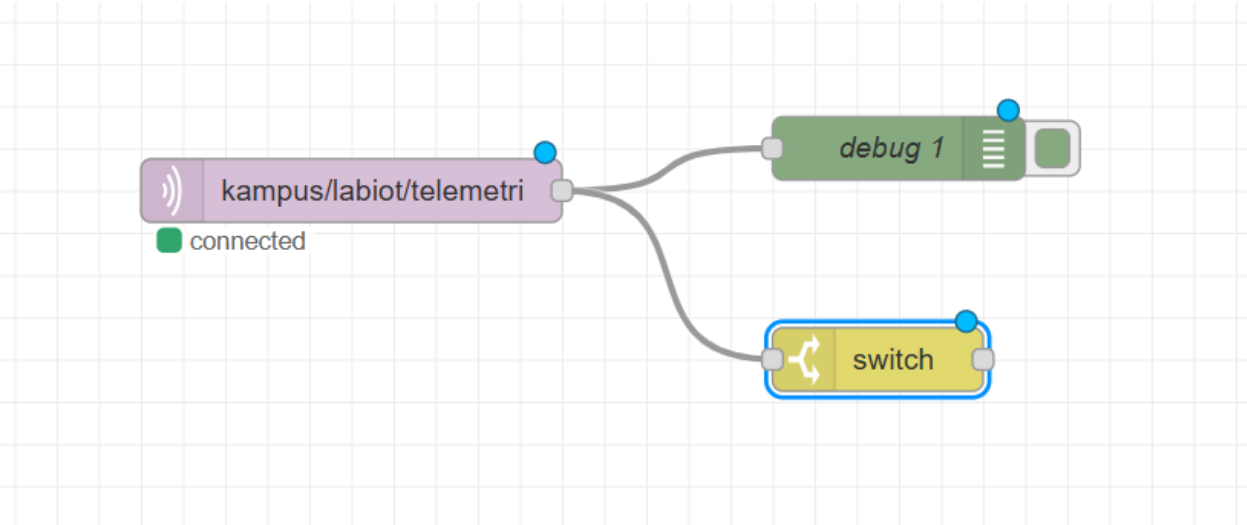
Menganalisis Aliran Data



Kesimpulan:

Praktikum ini berfokus pada integrasi alur data telemetri dua arah menggunakan antarmuka kanvas Node-RED untuk menangkap dan membedah aliran data dari mikrokontroler. Node-RED dikonfigurasi menggunakan node mqtt in dengan server broker lokal bernama mqtt pada port 1883 serta topik kampus/labiot/telemetri, di mana tipe output-nya diubah menjadi objek JSON terurai. Data telemetri tersebut kemudian dihubungkan ke node debug untuk memantau informasi masuk secara real-time setiap 4 detik pada panel sidebar, menampilkan parameter seperti identitas node, suhu, kelembapan, dan status LED setelah proses deploy berhasil terhubung.

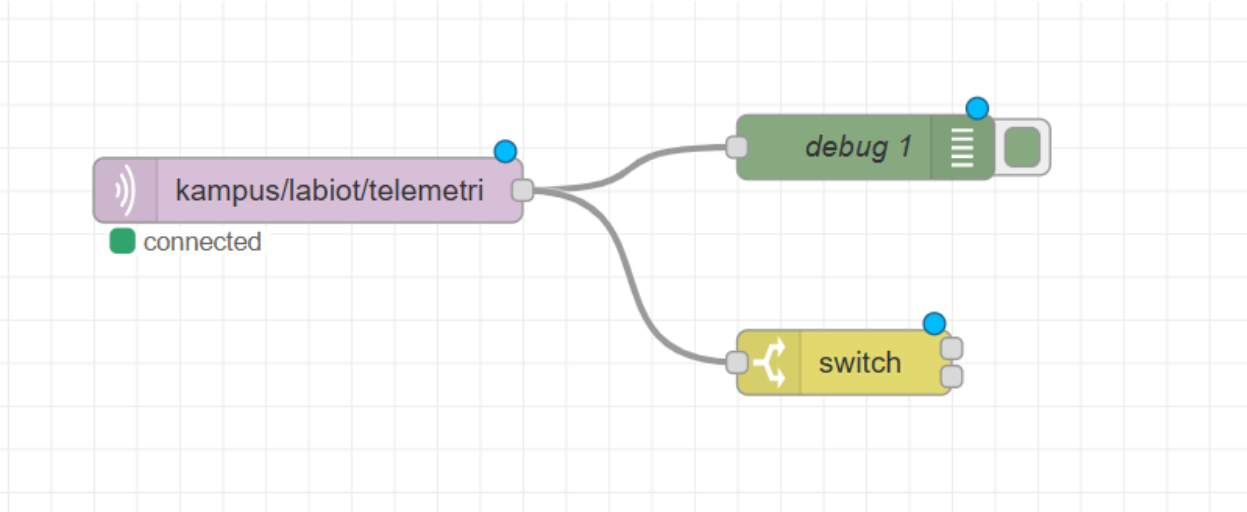
7.5.4 Praktikum 4: Implementasi Rule Engine Otomasi Aktuator Loop Tertutup



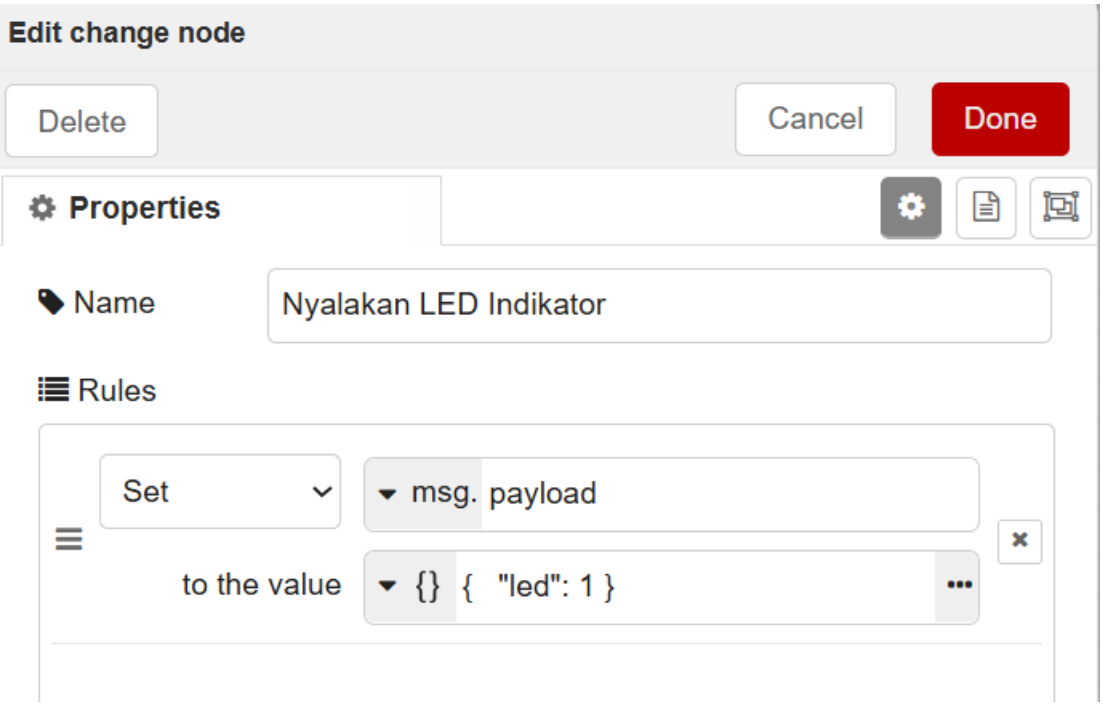
- Mengubah msg jadi payload.suhu dan membuat 2 aturan.



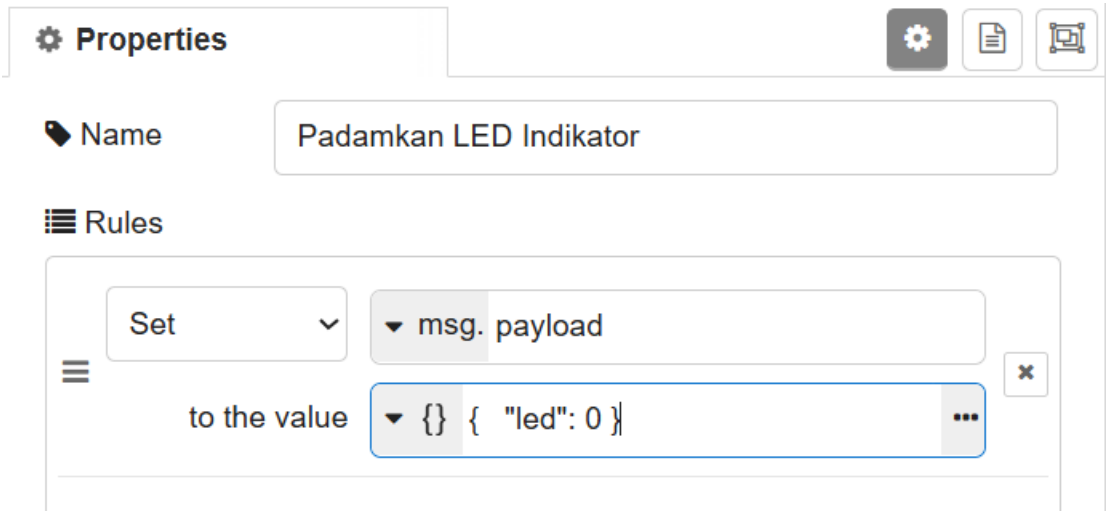
- Kini switch memliki 2 port



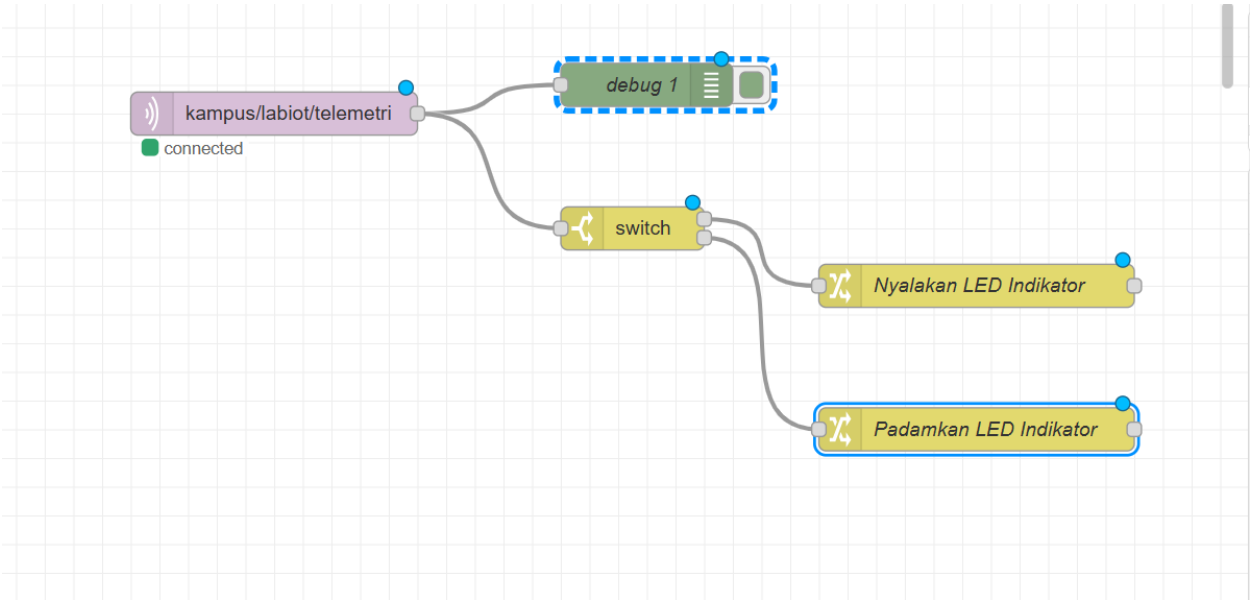
- Aturan untuk menyalakan led



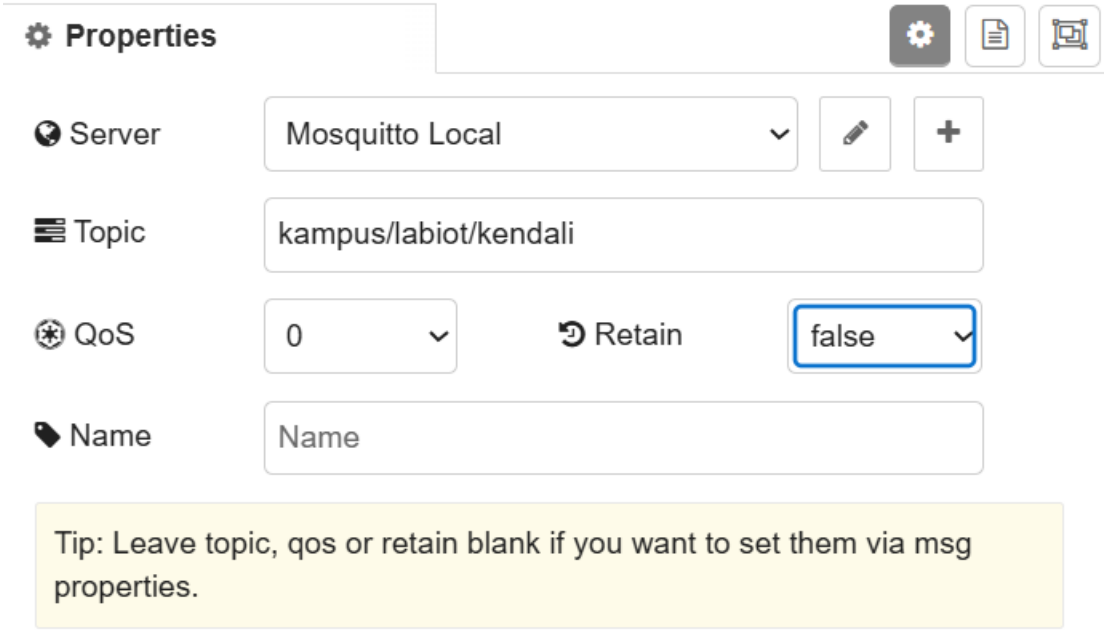
- Aturan untuk menyalakan led

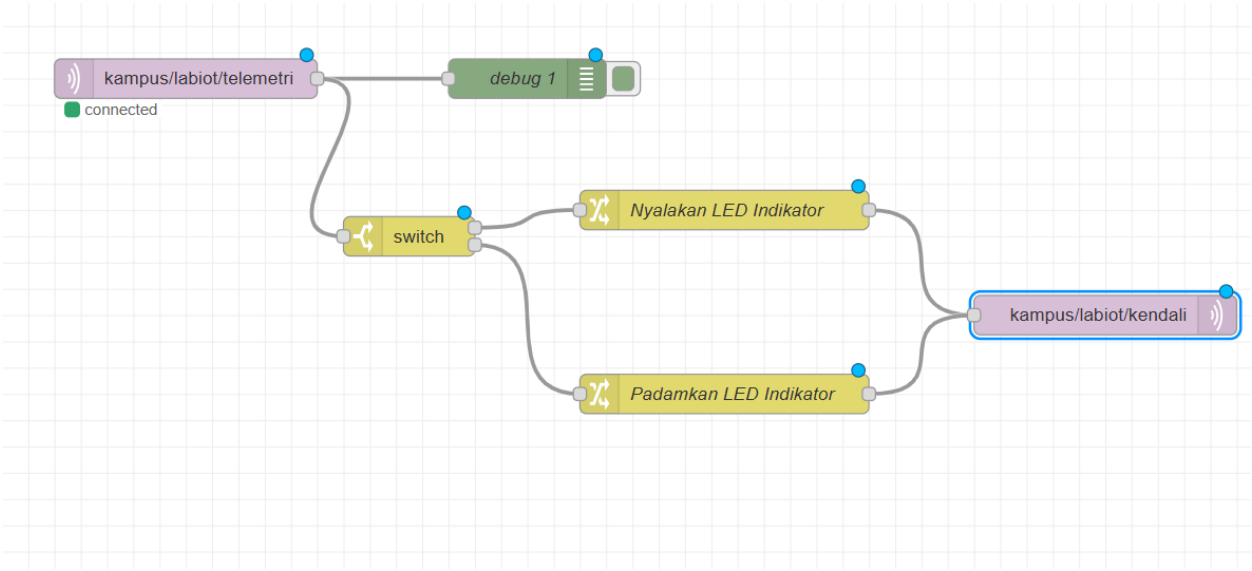


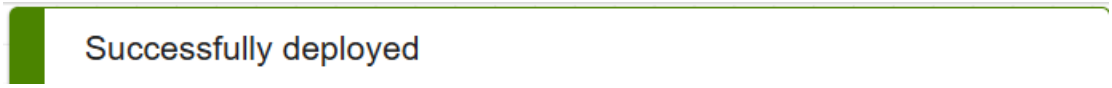
- Hasil



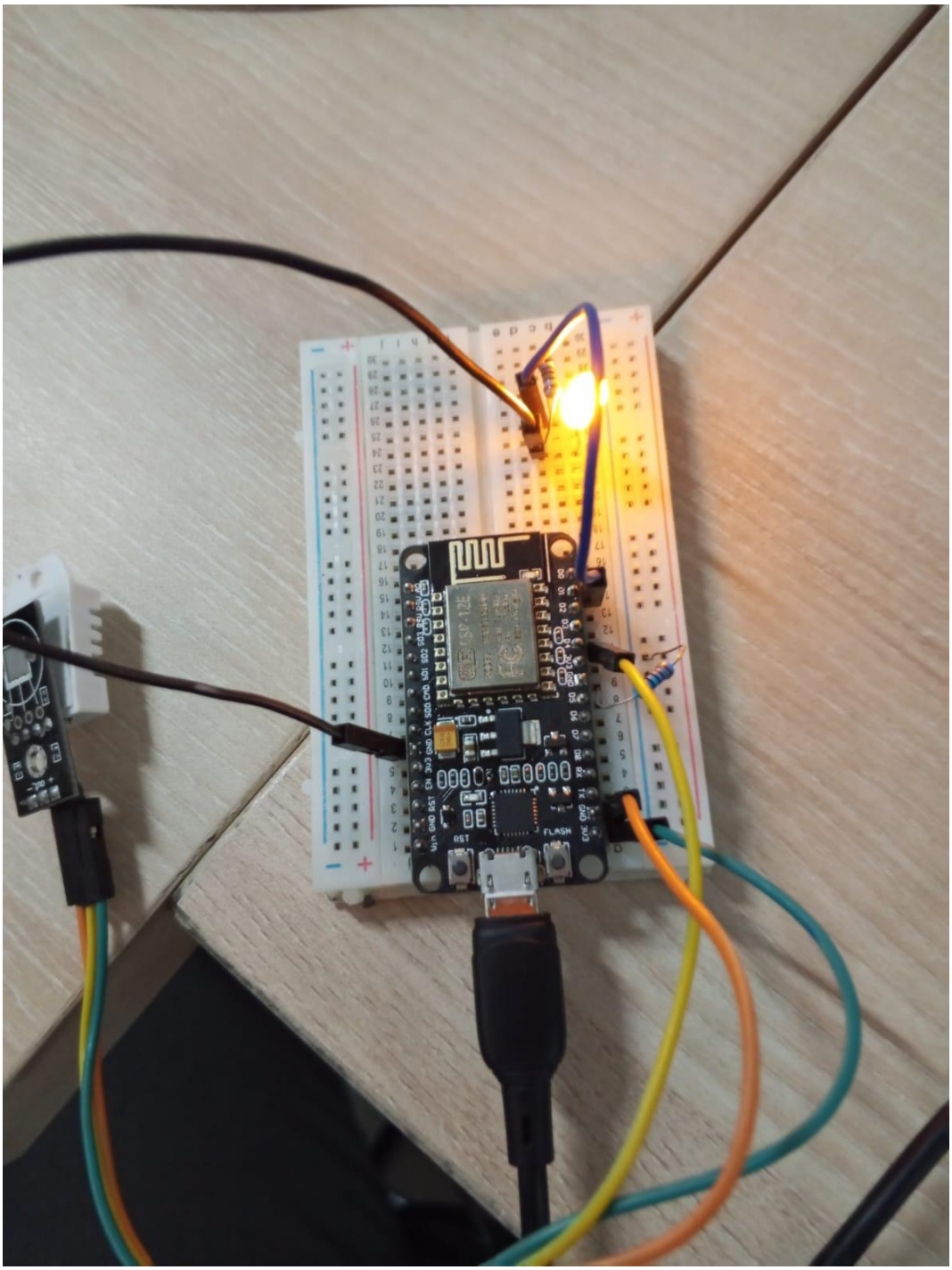
- Menerbitkan Instruksi Balik ke Broker (mqtt out)

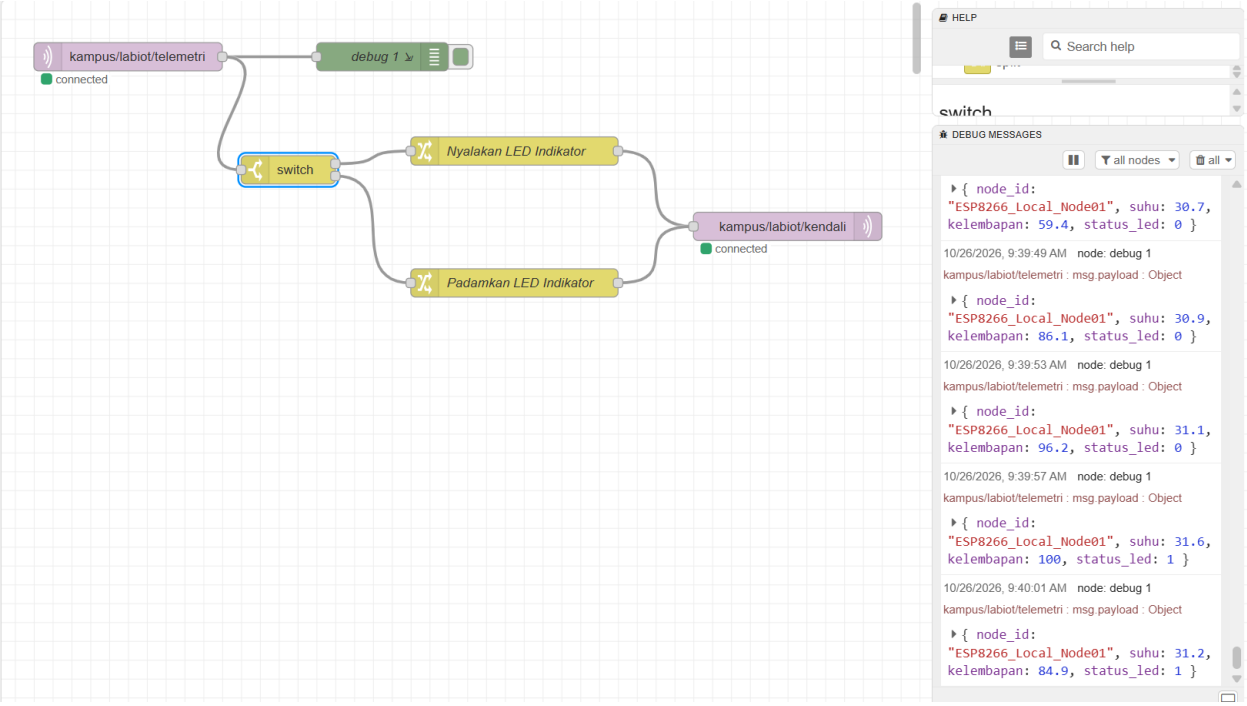






- Pengujian Fisik Sistem Otomasi





Kesimpulan:

Praktikum ini berfokus pada implementasi Closed-Loop Rule Engine otonom menggunakan Node-RED untuk mengendalikan aktuator LED secara otomatis berdasarkan ambang batas suhu. Sistem dirancang dengan menambahkan node switch yang mengevaluasi properti msg.payload.suhu menjadi dua kondisi, yaitu suhu di atas 31°C (port 1) dan suhu kurang dari atau sama dengan 31°C (port 2). Masing-masing jalur keluaran dihubungkan ke node change untuk mengubah muatan pesan (payload) menjadi format JSON berupa {"led": 1} untuk menyalakan indikator atau {"led": 0} untuk memadamkannya. Seluruh instruksi tersebut kemudian digabungkan dan diterbitkan kembali melalui node mqtt out dengan topik kampus/labiot/kendali menuju broker lokal setelah tombol deploy ditekan. Melalui pengujian fisik, sistem berhasil menjalankan mekanisme umpan balik secara real-time, di mana LED akan menyala otomatis saat sensor mendeteksi suhu melebihi 31°C dan kembali padam ketika suhu mendingin di bawah ambang batas tersebut.

7.6 Latihan

- Latihan 1 (Analisis Jaringan Internal Docker & Service Discovery): Pada Praktikum 3, di dalam konfigurasi node mqtt in pada Node-RED, kita mengisi alamat server broker dengan nama teks mqtt, bukan alamat IP lokal laptop (192.168.1.15) maupun localhost.
  i. Mengapa penulisan localhost pada Node-RED di dalam kontainer Docker akan menyebabkan kegagalan koneksi (Connection Refused)?

     Di dalam ekosistem Docker, setiap kontainer berjalan di dalam ruang isolasi jaringan (*network namespace*) tersendiri. Jika Node-RED menggunakan alamat localhost (atau 127.0.0.1), sistem akan mencoba mencari layanan broker MQTT di dalam kontainer Node-RED itu sendiri, bukan di dalam kontainer MQTT broker (mqtt-broker). Karena layanan Mosquitto berada di kontainer terpisah, maka koneksi akan ditolak (*connection refused*).

  ii. Di dalam ekosistem Docker, setiap kontainer berjalan di dalam ruang isolasi jaringan (*network namespace*) tersendiri. Jika Node-RED menggunakan alamat localhost (atau 127.0.0.1), sistem akan mencoba mencari layanan broker MQTT di dalam kontainer Node-RED itu sendiri, bukan di dalam kontainer MQTT broker (mqtt-broker). Karena layanan Mosquitto berada di kontainer terpisah, maka koneksi akan ditolak (*connection refused*).

     Di dalam jaringan Docker, kedua kontainer (mqtt-broker dan node-red) dihubungkan ke dalam satu jaringan virtual internal yang sama menggunakan *Bridge Network*, sehingga mereka dapat saling berkomunikasi secara privat. Selain itu, Docker menyediakan *Embedded DNS Server* yang secara otomatis memetakan setiap nama layanan (*service name*) yang didefinisikan dalam berkas deklaratif ke alamat IP internal kontainer yang bersangkutan. Ketika kontainer Node-RED mencoba memanggil host bernama mqtt, server DNS internal Docker akan menerjemahkannya secara transparan ke alamat IP kontainer mqtt-broker, sehingga komunikasi dapat terhubung dengan mulus tanpa harus menggunakan alamat IP fisik laptop.

- Latihan 2 (Penanganan Masalah Konektivitas Jaringan & Subnet Firewall): Seorang mahasiswa mencoba menghubungkan NodeMCU ESP8266 ke broker lokal di laptopnya. ESP8266 berhasil terhubung ke Wi-Fi dan mendapatkan alamat IP 192.168.43.102, sedangkan laptop memiliki alamat IP 192.168.1.15.
  i. Mengapa NodeMCU tidak akan pernah berhasil memublikasikan data ke broker Mosquitto pada kondisi tersebut?

NodeMCU dan laptop berada pada dua *subnet* atau segmen jaringan yang berbeda (NodeMCU berada di subnet 192.168.43.x, sedangkan laptop berada di subnet 192.168.1.x). Karena perbedaan segmen jaringan ini, komunikasi langsung tingkat *link-local* melalui protokol TCP/IP pada port 1883 tidak dapat dilakukan tanpa adanya perangkat router atau *gateway* yang merutekan lalu lintas data di antara kedua jaringan tersebut.

ii. Langkah teknis apa yang harus dilakukan mahasiswa tersebut agar kedua perangkat berada dalam domain *broadcast / subnet* yang sama?

Untuk memastikan kedua perangkat berada dalam domain *broadcast* atau *subnet* yang sama, mahasiswa tersebut dapat melakukan langkah-langkah teknis berikut:

- **Menghubungkan ke Access Point atau Wi-Fi yang Sama:** Pastikan laptop dan NodeMCU ESP8266 terhubung ke jaringan Wi-Fi, router, atau titik akses fisik yang persis sama sehingga mendapatkan alokasi segmen IP dari *gateway* yang seragam.
- **Menggunakan Hotspot Seluler yang Sama:** Hubungkan laptop dan NodeMCU ESP8266 secara bersamaan ke satu sumber *Wi-Fi Hotspot* (misalnya dari *smartphone* yang sama) agar kedua perangkat otomatis berada dalam satu jangkauan jaringan lokal dan subnet yang selaras.

- Latihan 3 (Implementasi Filter Histeresis pada Rule Engine):

Pada sistem kendali dua posisi (*Bang-Bang*), jika suhu berfluktuasi cepat di sekitar ambang batas 31.0°C(misal: 31.05°C → 30.95°C → 31.02°C), lampu LED indikator akan berkedip-kedip cepat (*flickering / hunting*) yang mengganggu stabilitas visual sistem.
Rancang modifikasi alur Node-RED menggunakan **function node** berbahasa JavaScript yang menerapkan logika **Histeresis**:

- LED baru menyala jika suhu > 31.5°C(msg.payload = {"led": 1}).
- LED baru padam jika suhu mendingin hingga < 30.0°C(msg.payload = {"led": 0}). Tuliskan kode JavaScript yang Anda masukkan ke dalam *Function Node* tersebut!

1. Tambahkan variabel global di bagian atas program (sebelum setup()):

```
// Batas histeresis
const float BATAS_ATAS = 31.5;   // LED menyala jika suhu > 31.5[cite: 2]
const float BATAS_BAWAH = 30.0;  // LED padam jika suhu < 30.0[cite: 2]
bool ledState = false;           // Menyimpan status LED terakhir[cite: 2]
```

2. Ubah bagian pembacaan sensor di dalam fungsi loop():

```
float t = dht.readTemperature();
float h = dht.readHumidity();

if (!isnan(t) && !isnan(h)) {

  // ---- Logika Histeresis ----
  if (t > BATAS_ATAS) {
    ledState = true;
  } else if (t < BATAS_BAWAH) {
    ledState = false;
  }
  // Jika 30.0 <= t <= 31.5: status LED tidak berubah[cite: 2]
  digitalWrite(ledPin, ledState ? HIGH : LOW);

  StaticJsonDocument<256> docOut;
  docOut["node_id"]    = clientId;
  docOut["suhu"]       = serialized(String(t, 2));
  docOut["kelembapan"] = serialized(String(h, 2));
  docOut["status_led"] = digitalRead(ledPin);
```

- Latihan 4 (Analisis Mekanisme Volume Mount & Persistensi Data): Pada berkas docker-compose.yml, terdapat pemetaan direktori - ./nodered:/data dan - ./mosquitto/data:/mosquitto/data.
  i. Apa yang akan terjadi pada alur kanvas visual Node-RED yang telah Anda buat jika kontainer dimatikan dengan perintah docker compose down, kemudian dinyalakan kembali, seandainya baris volume tersebut dihapus dari docker-compose.yml?

     Jika baris pemetaan volume tersebut dihapus, seluruh alur kanvas visual, konfigurasi node, serta data yang tersimpan di dalam direktori internal kontainer Node-RED akan ikut terhapus (*hilang*) ketika kontainer dimatikan dengan perintah docker compose down. Akibatnya, saat kontainer dihidupkan kembali (docker compose up), Node-RED akan kembali seperti kondisi awal saat pertama kali dipasang (bersih tanpa kanvas dan alur kerja yang sebelumnya telah dibuat).

ii. Jelaskan perbedaan mendasar antara *Bind Mount* dan *Named Volume* pada Docker!

- **Bind Mount:** Mengandalkan direktori atau berkas yang ada pada sistem host (komputer fisik Anda) untuk dipetakan secara langsung ke dalam kontainer. Jalur path pada host harus ditentukan secara eksplisit (misalnya ./nodered:/data), sehingga perubahan pada data di dalam kontainer dapat langsung terlihat atau disimpan di direktori fisik host tersebut.
- **Named Volume:** Dikelola sepenuhnya oleh Docker di dalam area penyimpanan internal host yang khusus dikhususkan untuk Docker (biasanya terletak di /var/lib/docker/volumes/ pada Linux). Pengguna tidak perlu menentukan jalur fisik secara spesifik di host, cukup memberikan nama volumenya saja, sehingga dinilai lebih aman, portabel, dan terisolasi dari sistem berkas utama host.

7.7 Tugas: Pengamanan Server IoT Industri (MQTT Authentication & Access Control)

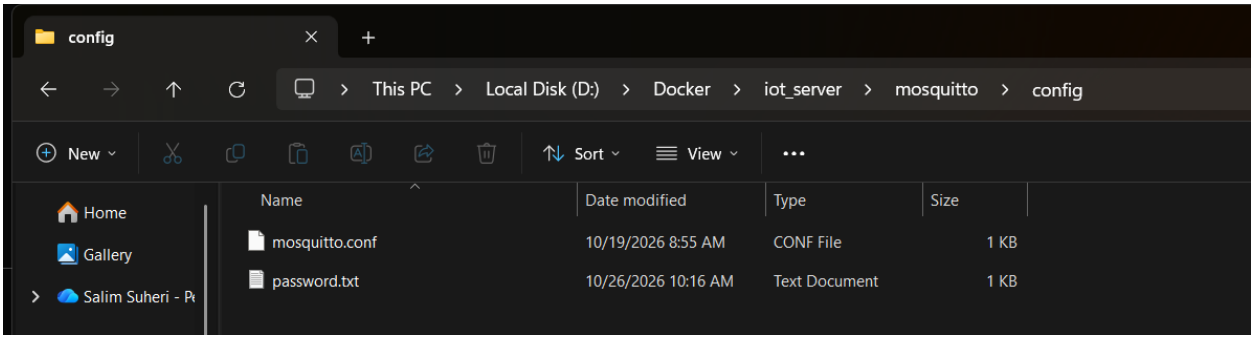
1. Membuat Berkas Kredensial Terenkripsi di Dalam Kontainer

```
C:\Users\Salim>docker exec -it mqtt-broker mosquitto_passwd -c /mosquitto/config/password.txt admin
Password:
Reenter password:
Adding password for user admin

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug mqtt-broker
    Learn more at https://docs.docker.com/go/debug-cli/

C:\Users\Salim>
```

File password di folder config



2. Memperbarui Konfigurasi Kebijakan Mosquitto

```
# Perizinan Akses Awal (Tanpa Password)
allow_anonymous false
password_file /mosquitto/config/password.txt
```

3. Memuat Ulang Layanan Broker

```
C:\Users\Salim>docker restart mqtt-broker
mqtt-broker

C:\Users\Salim>
```

4. Analisis Dampak Kegagalan Sistem (Observasi Masalah)

```
10:28:49.388 -> [MQTT] Menghubungkan ke Broker Lokal (10.155.147.183)... Gagal (rc=-2). Coba lagi dalam 4 detik...
10:28:53.393 -> [MQTT] Menghubungkan ke Broker Lokal (10.155.147.183)... Gagal (rc=-2). Coba lagi dalam 4 detik...
10:28:57.388 -> [MQTT] Menghubungkan ke Broker Lokal (10.155.147.183)... Gagal (rc=-2). Coba lagi dalam 4 detik...
10:29:01.417 -> [MQTT] Menghubungkan ke Broker Lokal (10.155.147.183)... Gagal (rc=-2). Coba lagi dalam 4 detik...
10:29:05.392 -> [MQTT] Menghubungkan ke Broker Lokal (10.155.147.183)... Gagal (rc=-2). Coba lagi dalam 4 detik...
10:29:09.411 -> [MQTT] Menghubungkan ke Broker Lokal (10.155.147.183)... Gagal (rc=-2). Coba lagi dalam 4 detik...
10:29:13.397 -> [MQTT] Menghubungkan ke Broker Lokal (10.155.147.183)... Gagal (rc=-2). Coba lagi dalam 4 detik...
10:29:17.394 -> [MQTT] Menghubungkan ke Broker Lokal (10.155.147.183)... Gagal (rc=-2). Coba lagi dalam 4 detik...
10:29:21.435 -> [MQTT] Menghubungkan ke Broker Lokal (10.155.147.183)... Gagal (rc=-2). Coba lagi dalam 4 detik...
10:29:25.422 -> [MQTT] Menghubungkan ke Broker Lokal (10.155.147.183)... Gagal (rc=-2). Coba lagi dalam 4 detik...
10:29:29.422 -> [MQTT] Menghubungkan ke Broker Lokal (10.155.147.183)... Gagal (rc=-2). Coba lagi dalam 4 detik...
10:29:33.422 -> [MQTT] Menghubungkan ke Broker Lokal (10.155.147.183)... Gagal (rc=-2). Coba lagi dalam 4 detik...
```

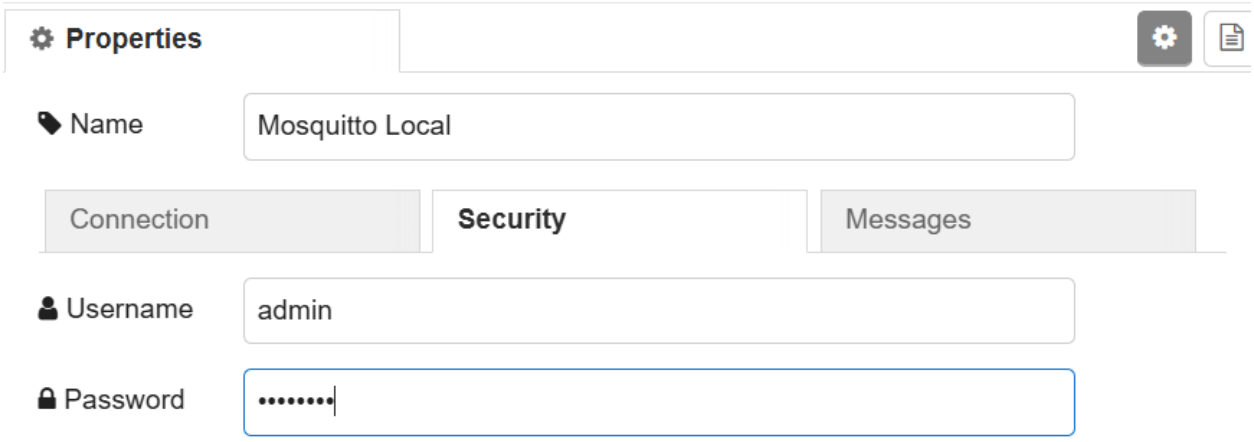
Kode status rc=5 merupakan representasi standar MQTT untuk 'Connection Refused: Not Authorized

5. Rekayasa Perbaikan Sistem
   Pemutakhiran Skrip Program NodeMCU ESP8266

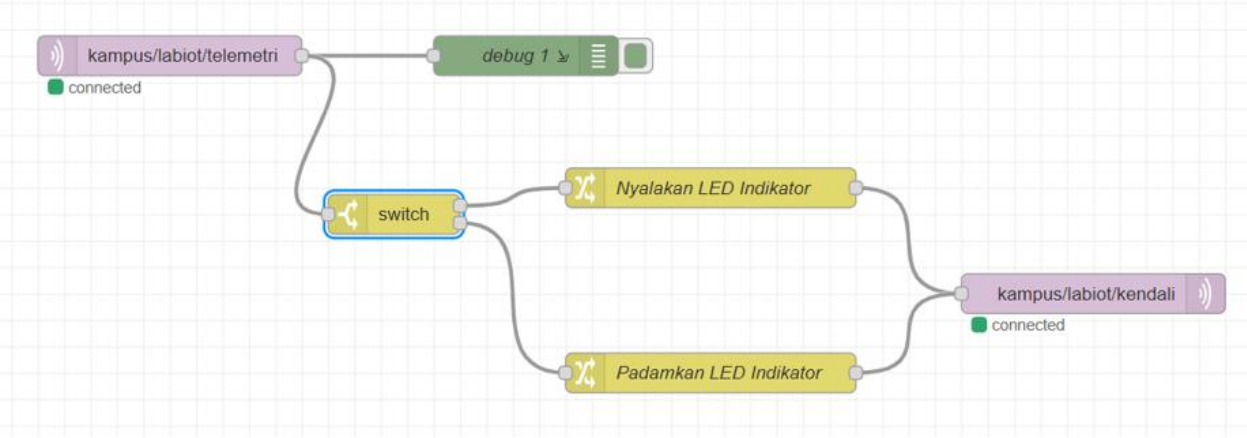
```
// Format: client.connect(clientId, username, password)
  if (client.connect(clientId, "admin", "salim123")) {
    Serial.println(" Terhubung dengan Autentikasi Sah!");
    // Segera daftarkan subskripsi topik kendali
    client.subscribe(topicSubCmd);
    Serial.printf("[MQTT] Berlangganan ke topik: %s\n", topicSubCmd);
  } else {
    Serial.printf(" Gagal (rc=%d). Coba lagi dalam 4 detik...\n", client.state());
    delay(4000);
  }
```

Pemutakhiran Konfigurasi Kredensial pada Node-RED

nama broker Mosquitto Local dan username admin dan password rahasia123



Hasil nya



Kesimpulan:

Berdasarkan seluruh rangkaian praktikum, sistem IoT lokal berhasil dibangun menggunakan Docker Compose untuk komunikasi antar-kontainer yang stabil. Mikrokontroler NodeMCU ESP8266 sukses mengirim data telemetri sensor DHT dan menerima kendali LED melalui broker lokal. Node-RED juga telah dikonfigurasi sebagai rule engine dengan penerapan logika histeresis guna mencegah flickering akibat fluktuasi suhu. Terakhir, aspek keamanan industri berhasil diterapkan dengan menonaktifkan akses anonim dan menggunakan kredensial terenkripsi (admin / rahasia123), di mana penolakan koneksi berstatus rc=5 berhasil dipulihkan dengan menyematkan username dan password yang valid pada program ESP8266 maupun Node-RED.